

WORKBENCH IT · POC INTERNO

SecondBrain

Memoria aumentada privada,
100 % local.

Indexa, procesa y consulta tu actividad personal y profesional – conversaciones de WhatsApp, audios, documentos, imágenes– usando solo modelos locales. Nada sale del equipo.

LOCAL

LLMs, embeddings y transcripción en GPU propia. Costo recurrente \$0.

VAULT

Los archivos crudos se guardan, no solo se procesan. Re-procesables a futuro.

MODULAR

Cada etapa del pipeline puede crecer o reemplazarse sin tocar el resto.

PRIVADO

Ninguna API externa. Diseñado para datos personales sin compartir.

Versión: overview técnico al 2026-05-17

Hardware: i7-10ma · 32 GB RAM · RTX 3070 Ti 8 GB VRAM · Windows 11 + WSL2 + Docker Desktop

Doc complementario: pipeline.pdf – foto técnica detallada con métricas

Arquitectura en una página

Cuatro capas independientes. Cada ítem (mensaje, audio, imagen, documento) fluye de ingesta a almacenamiento y queda disponible para el chat de Q&A.

1. INGESTA

Bridge live

whatsapp-web.js captura mensajes nuevos en tiempo real

Import histórico

Parser de exports .txt de WhatsApp

Uploads manuales

Audios, PDFs, DOCX, imágenes desde el panel



2. PROCESAMIENTO – WORKER CONTINUO, 5 ETAPAS

transcribe

Whisper Large V3
CPU

extract

pdfplumber / docx / xlsx
CPU

caption

qwen3-vl:8b
GPU · ventana 02-06

embed

bge-m3 · 1024 dim
GPU

tagger

qwen3:8b
GPU · facts + promesas



3. ALMACENAMIENTO

PostgreSQL 16

5 schemas (core, media, processing, analytics, audit). Modelo de entidades, jobs, attachments meta.

Qdrant

Vectores 1024-dim de mensajes y facts. Búsqueda semántica con filtros.

MinIO

Vault de archivos crudos (audios .opus, imágenes, PDFs). Dedup por SHA-256.



4. Q&A (CHAT)

Query understanding

qwen3:8b extrae personas y expande la query

Entity resolution

Match a personas canónicas → filtro persona_id

Retrieval híbrido

Busca en messages + facts, mergea por score

Generación con citas

qwen3:8b responde citando los fragmentos

Stack & decisiones

Todo orquestado por Docker Compose. Modelos servidos por Ollama. Panel de control nativo en escritorio (PySide6) para no depender del browser.

Servicios

Servicio	Imagen	Para qué	Recursos
postgres	pgvector/pgvector:pg16	Modelo de dominio, jobs, metadata de archivos	CPU + RAM
qdrant	qdrant/qdrant	Vectores de mensajes y facts (1024 dim, cosine)	CPU + RAM
minio	minio/minio	Object storage (Vault de binarios)	CPU + disco
ollama	ollama/ollama	Servidor LLM local (chat, embeddings, visión)	GPU
whisper	onerahmet/.../whisper-asr	Transcripción de audios (large-v3-turbo, CPU)	CPU
backend	Python 3.12 + FastAPI + uv	API + worker continuo	CPU + RAM
frontend	Streamlit	Páginas de validación, debugging, uploads	CPU
bridge	Node + whatsapp-web.js	Captura WhatsApp en vivo, descarga media	CPU

Modelos en uso

Modelo	VRAM	RoI
qwen3:8b	5.2 GB	Chat principal + tagger. Caliente 30 min con OLLAMA_KEEP_ALIVE
bge-m3	1.2 GB	Embeddings (mensajes y facts). Convive en VRAM con el chat
qwen3-vl:8b	6.1 GB	Visión (OCR, caption, entidades). Solo se carga 02-06 hs
Whisper large-v3-turbo	CPU	Transcripción de audios (4-bit). Decisión consciente: libera GPU para el chat

Decisiones que definen el diseño

100 % local, sin nube PRIVACIDAD LLMs, embeddings, transcripción, OCR y vector DB corren en el equipo. Sin	Procesamiento tiered en cola ARQUITECTURA
---	---

facturación recurrente. Sin compartir datos con terceros.

Un worker continuo dentro del backend drena 5 etapas cada 30 s. Auto-encadenado: embed → tagger → re-embed.

VRAM-aware: 8 GB son el techo

HARDWARE

Cada modelo se elige para que chat + embedding convivan sin swap. qwen3:8b + bge-m3 = 7.2 GB. Visión va de noche.

Calidad > LLM final

PIPELINE

El tagger extrae facts, promesas y transacciones como texto pre-estructurado. Eso embebido rankea mucho mejor que mensajes crudos.

Vault crudo, no índice

STORAGE

Audios, imágenes y PDFs originales se guardan en MinIO con dedup por SHA-256. Reprocesable cuando aparezcan modelos mejores.

Entity resolution canónico

COLUMNA VERTEBRAL

"Esteban", "Esteban K" y "+54 9 223..." resuelven a la misma persona. Habilita queries como "última vez que hablé con X".

Mapa de código & roadmap

Repositorio organizado por responsabilidad. Cuatro grandes piezas: backend, frontend, bridge, panel de escritorio.

```
# Repositorio
secondbrain/
├── backend/                                     Python 3.12 · FastAPI · SQLAlchemy 2 · uv
│   ├── app/
│   │   ├── config.py                         pydantic-settings (modelo, ventanas, batch)
│   │   ├── main.py                          lifespan: arranca el worker continuo
│   │   ├── models/                         core, tagging, media, processing
│   │   ├── routers/                       bridge, chat, tagger, embeddings,
│   │   │                                       transcribe, extract, images, panel,
│   │   │                                       worker, imports, contacts, conversations
│   │   └── services/
│   │       ├── queue_worker.py             □ drena las 5 etapas, async
│   │       ├── tagger.py                   qwen3:8b → facts/promesas/transacciones
│   │       ├── embedder.py                 bge-m3 → Qdrant messages + facts
│   │       ├── retriever.py                búsqueda híbrida + filtros (persona, fecha)
│   │       ├── chat.py                     query expansion + generación con citas
│   │       ├── ollama_client.py            wrapper con flag force_cpu (reserva)
│   │       ├── qdrant_client.py            collections, upsert, search
│   │       ├── minio_client.py             Vault: raw / derived / exports
│   │       ├── whisper_client.py, whatsapp_parser.py, vcard_parser.py
│   └── alembic/versions/                   4 migraciones acumuladas
├── bridge/                                  Node.js · whatsapp-web.js
│   └── (sesión persistente en data/bridge/, POST al backend)
├── frontend/                               Streamlit · validación y uploads
│   └── pages/1..13_*.py                    dashboard, import, contactos, chat, etc.
├── panel/                                  □ PySide6 · escritorio
│   ├── secondbrain_panel/
│   │   ├── main_window.py                  7 tabs: sistema, servicios, worker,
│   │   │                                       colas, chats, tagger, configuración
│   │   ├── docker_client.py               wrapper docker compose
│   │   └── api_client.py                   httpx async al backend
├── docker-compose.yml                      8 servicios + init jobs
└── docs/                                  sprints.md · architecture.md · pipeline.md
```

Roadmap – dónde estamos parados

SPRINT 0-1 □

Setup, modelos,
schema base, parser
WhatsApp

SPRINT 2-3 □

Bridge live,
contactos, tagger
end-to-end

SPRINT 4 □

Embeddings,
retriever, chat
funcional

SPRINT 5-7 □

Imágenes,
documentos, audios

WORKER CONTINUO □

Drena 5 colas async
cada 30 s

PANEL ESCRITORIO □

PySide6 +
endpoints /api/panel

MIGRACIÓN BGE-M3 □

A/B con datos reales
· 2026-05-16

SPRINT 8 →

Conector Gmail (IMAP
+ parser MIME)

Lo que ya hace bien

- Ingesta automática de WhatsApp en vivo
- Pipeline de 5 etapas con re-encadenado
- Chat con citas en ~2 s (warm)
- Entity resolution para personas canónicas
- Panel de orquestación nativo

Próximas decisiones a discutir

- Conector Gmail (Sprint 8)
- Memoria estructurada / knowledge graph
- Briefings proactivos (¿qué tengo hoy?)
- UI productiva (reemplazar Streamlit)
- ¿Upgrade de GPU a 16 GB?